МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №7

Специальность ПО-13

Выполнил:

Заяц Н.Д.

студент группы ПО-13

Проверил:

Крощенко А.А.

13.02.2026

Брест 2026

Лабораторная работа 7

Цель работы: освоить возможности языка программирования Python в разработке оконных приложений

Требования к оформлению отчета

Отчет по лабораторной работе должен содержать следующие разделы (примеры оформления отчетов можно найти в папке с заданиями):

1) Изложение цели работы.
2) Задание по лабораторной работе с описанием своего варианта.
3) Спецификации ввода-вывода программы.
4) Текст программы (кратко).
5) Выводы по проделанной работе.

Задание 1. Построение графических примитивов и надписей

Требования к выполнению

• Реализовать соответствующие классы, указанные в задании;

• Организовать ввод параметров для создания объектов (использовать экранные компоненты);

• Осуществить визуализацию графических примитивов

Важное замечание: должна быть предусмотрена возможность приостановки выполнения визуализации, изменения параметров «на лету» и снятия скриншотов с сохранением в текущую активную директорию.

Для всех динамических сцен необходимо задавать параметр скорости!

4) Изобразить разносторонний треугольник, вращающийся в плоскости формы вокруг своего центра тяжести.

```
"""Task 1, variant 4: rotating scalene triangle."""

# pylint: disable=too-few-public-methods,duplicate-code

from __future__ import annotations

import math
from dataclasses import dataclass
from pathlib import Path
from tkinter import Button, Canvas, Entry, Frame, Label, StringVar, Tk, messagebox

WINDOW_WIDTH = 1000
WINDOW_HEIGHT = 700
CONTROL_WIDTH = 290
CANVAS_WIDTH = WINDOW_WIDTH - CONTROL_WIDTH
CANVAS_HEIGHT = WINDOW_HEIGHT
ANIMATION_DELAY_MS = 16
```

```
SCREENSHOT_PREFIX = "rotating_triangle"

BACKGROUND_COLOR = "#1f2328"
PANEL_COLOR = "#2d333b"
TEXT_COLOR = "#f0f3f6"
TRIANGLE_COLOR = "#ffcc66"
OUTLINE_COLOR = "#f08c00"
CENTER_COLOR = "#4dabf7"


@dataclass(frozen=True)
class Point:
    """A point in the form plane."""

    x: float
    y: float


@dataclass(frozen=True)
class TriangleParameters:
    """Parameters that define triangle view and motion."""

    point_a: Point
    point_b: Point
    point_c: Point
    speed: float
    fill_color: str
    outline_color: str


class ScaleneTriangle:
    """Scalene triangle that rotates around its centroid."""

    def __init__(self, parameters: TriangleParameters) -> None:
        self.parameters = parameters
        self.angle = 0.0

    @property
    def centroid(self) -> Point:
        """Return the center of gravity."""
        return Point(
            (self.parameters.point_a.x + self.parameters.point_b.x +
self.parameters.point_c.x) / 3,
            (self.parameters.point_a.y + self.parameters.point_b.y +
self.parameters.point_c.y) / 3,
        )

    def update_parameters(self, parameters: TriangleParameters) -> None:
        """Apply new triangle parameters without resetting the current angle."""
```

```
    self.parameters = parameters

def rotate(self) -> None:
    """Advance the rotation angle."""
    self.angle = (self.angle + self.parameters.speed) % 360

def rotated_points(self) -> list[Point]:
    """Return triangle vertices after rotation around the centroid."""
    center = self.centroid
    angle_radians = math.radians(self.angle)
    cos_angle = math.cos(angle_radians)
    sin_angle = math.sin(angle_radians)
    points = [
        self.parameters.point_a,
        self.parameters.point_b,
        self.parameters.point_c,
    ]

    rotated = []
    for point in points:
        dx = point.x - center.x
        dy = point.y - center.y
        rotated.append(
            Point(
                center.x + dx * cos_angle - dy * sin_angle,
                center.y + dx * sin_angle + dy * cos_angle,
            )
        )

    return rotated

def draw(self, canvas: Canvas) -> None:
    """Draw the rotated triangle and its center of gravity."""
    points = self.rotated_points()
    flat_points = [coordinate for point in points for coordinate in (point.x, point.y)]
    center = self.centroid

    canvas.create_polygon(
        flat_points,
        fill=self.parameters.fill_color,
        outline=self.parameters.outline_color,
        width=3,
    )
    canvas.create_oval(
        center.x - 5,
        center.y - 5,
        center.x + 5,
        center.y + 5,
```

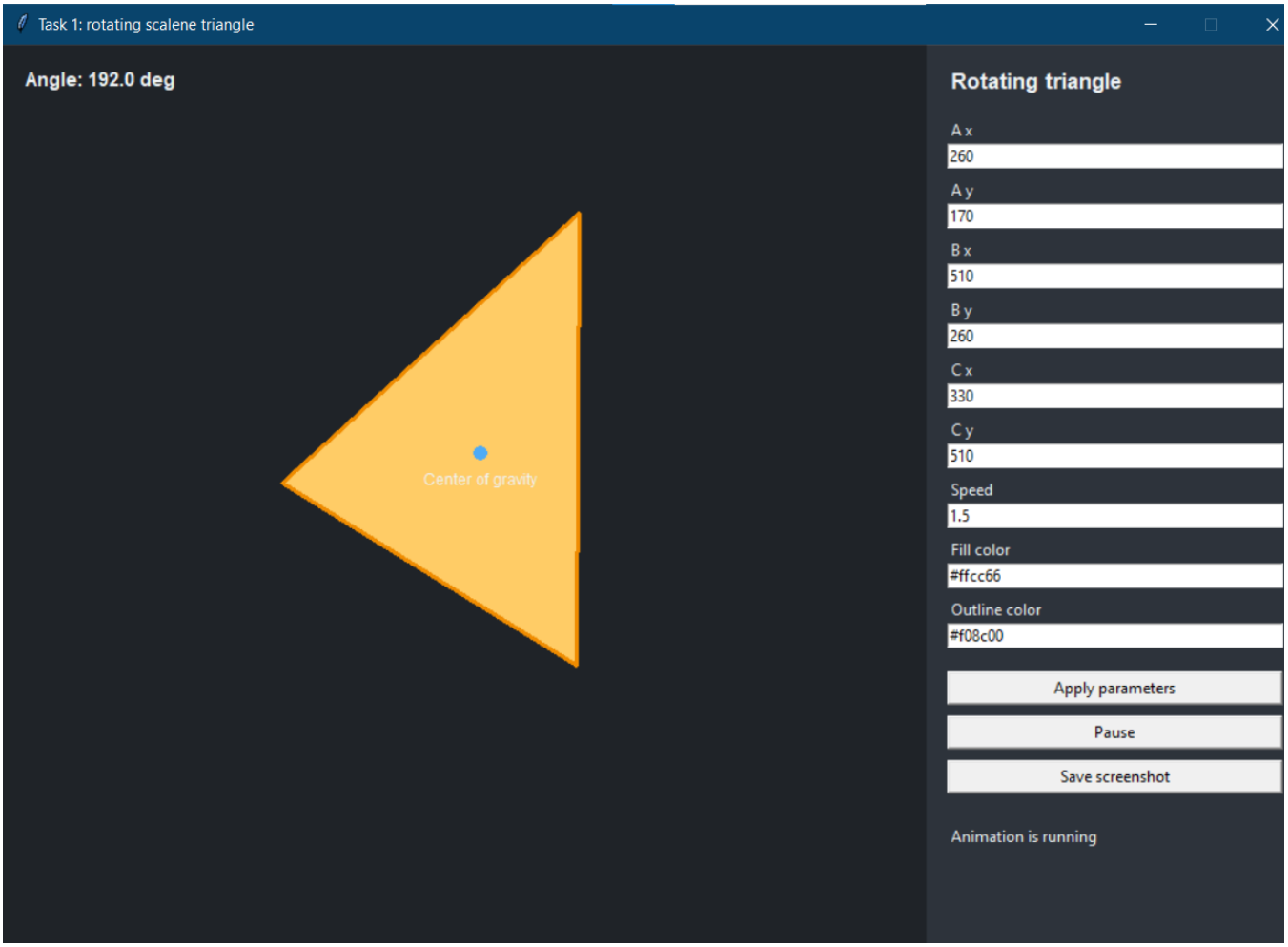
```
            fill=CENTER_COLOR,
            outline=CENTER_COLOR,
        )
        canvas.create_text(
            center.x,
            center.y + 20,
            text="Center of gravity",
            fill=TEXT_COLOR,
            font=("Arial", 9),
        )


class TriangleApplication:
    """Window application for the rotating triangle task."""

    def __init__(self) -> None:
        self.root = Tk()
        self.root.title("Task 1: rotating scalene triangle")
        self.root.geometry(f"{WINDOW_WIDTH}x{WINDOW_HEIGHT}")
        self.root.resizable(False, False)

        self.entries: dict[str, Entry] = {}
        self.status_text = StringVar(value="Animation is running")
        self.pause_button_text = StringVar(value="Pause")
        self.is_paused = False

        self.triangle = ScaleneTriangle(
            TriangleParameters(
                point_a=Point(260, 170),
                point_b=Point(510, 260),
                point_c=Point(330, 510),
                speed=1.5,
                fill_color=TRIANGLE_COLOR,
                outline_color=OUTLINE_COLOR,
            )
        )

        self.canvas = Canvas(
            self.root,
            width=CANVAS_WIDTH,
            height=CANVAS_HEIGHT,
            bg=BACKGROUND_COLOR,
            highlightthickness=0,
        )
        self.canvas.pack(side="left", fill="both")

        control_panel = Frame(self.root, width=CONTROL_WIDTH,
bg=PANEL_COLOR, padx=16, pady=16)
```

```
        control_panel.pack(side="right", fill="y")
        control_panel.pack_propagate(False)

        self._create_controls(control_panel)
        self._draw_scene()
        self._animate()

    def _create_controls(self, control_panel: Frame) -> None:
        """Create screen controls for changing triangle parameters."""
        Label(
            control_panel,
            text="Rotating triangle",
            bg=PANEL_COLOR,
            fg=TEXT_COLOR,
            font=("Arial", 13, "bold"),
        ).pack(anchor="w", pady=(0, 14))

        default_values = {
            "A x": "260",
            "A y": "170",
            "B x": "510",
            "B y": "260",
            "C x": "330",
            "C y": "510",
            "Speed": "1.5",
            "Fill color": TRIANGLE_COLOR,
            "Outline color": OUTLINE_COLOR,
        }

        for label_text, value in default_values.items():
            self._add_entry(control_panel, label_text, value)

        Button(control_panel, text="Apply parameters",
command=self.apply_parameters).pack(
            fill="x",
            pady=(12, 4),
        )
        Button(
            control_panel,
            textvariable=self.pause_button_text,
            command=self.toggle_pause,
        ).pack(fill="x", pady=4)
        Button(control_panel, text="Save screenshot",
command=self.save_screenshot).pack(
            fill="x",
            pady=4,
        )
```

```
        Label(
            control_panel,
            textvariable=self.status_text,
            bg=PANEL_COLOR,
            fg=TEXT_COLOR,
            wraplength=245,
            justify="left",
        ).pack(anchor="w", pady=(18, 0))

    def _add_entry(self, control_panel: Frame, label_text: str, value: str) -> None:
        """Add a labeled input field."""
        Label(control_panel, text=label_text, bg=PANEL_COLOR,
fg=TEXT_COLOR).pack(anchor="w")
        entry = Entry(control_panel)
        entry.insert(0, value)
        entry.pack(fill="x", pady=(0, 6))
        self.entries[label_text] = entry

    def apply_parameters(self) -> None:
        """Read parameters from screen controls and update the triangle."""
        try:
            parameters = self._read_parameters()
            self._validate_scalene_triangle(parameters)
            self.triangle.update_parameters(parameters)
            self._draw_scene()
            self.status_text.set("Parameters applied")
        except ValueError as error:
            messagebox.showerror("Input error", str(error))

    def toggle_pause(self) -> None:
        """Pause or resume visualization."""
        self.is_paused = not self.is_paused
        if self.is_paused:
            self.pause_button_text.set("Resume")
            self.status_text.set("Animation is paused")
        else:
            self.pause_button_text.set("Pause")
            self.status_text.set("Animation is running")

    def save_screenshot(self) -> None:
        """Save the canvas image to the active directory."""
        filename = self._next_screenshot_name()
        self.canvas.postscript(file=filename, colormode="color")
        self.status_text.set(f"Screenshot saved: {filename}")

    def run(self) -> None:
        """Start the application."""
        self.root.mainloop()
```

```
    def _animate(self) -> None:
        """Animation loop."""
        if not self.is_paused:
            self.triangle.rotate()
            self._draw_scene()

        self.root.after(ANIMATION_DELAY_MS, self._animate)

    def _draw_scene(self) -> None:
        """Draw the current scene."""
        self.canvas.delete("all")
        self.canvas.create_text(
            18,
            18,
            text=f"Angle: {self.triangle.angle:.1f} deg",
            anchor="nw",
            fill=TEXT_COLOR,
            font=("Arial", 11, "bold"),
        )
        self.triangle.draw(self.canvas)

    def _read_parameters(self) -> TriangleParameters:
        """Read and validate values from input fields."""
        point_a = Point(float(self.entries["A x"].get()), float(self.entries["A y"].get()))
        point_b = Point(float(self.entries["B x"].get()), float(self.entries["B y"].get()))
        point_c = Point(float(self.entries["C x"].get()), float(self.entries["C y"].get()))
        speed = float(self.entries["Speed"].get())
        fill_color = self.entries["Fill color"].get().strip()
        outline_color = self.entries["Outline color"].get().strip()

        if speed < 0:
            raise ValueError("Speed must not be negative")

        for point_name, point in {"A": point_a, "B": point_b, "C": point_c}.items():
            if not 0 <= point.x <= CANVAS_WIDTH or not 0 <= point.y <=
CANVAS_HEIGHT:
                raise ValueError(f"Point {point_name} must be inside the drawing area")

        if not fill_color or not outline_color:
            raise ValueError("Colors must not be empty")

        return TriangleParameters(
            point_a=point_a,
            point_b=point_b,
            point_c=point_c,
            speed=speed,
            fill_color=fill_color,
```

```
            outline_color=outline_color,
        )

    @staticmethod
    def _validate_scalene_triangle(parameters: TriangleParameters) -> None:
        """Check that the triangle exists and has three different side lengths."""
        points = [parameters.point_a, parameters.point_b, parameters.point_c]
        area = (
            abs(
                points[0].x * (points[1].y - points[2].y)
                + points[1].x * (points[2].y - points[0].y)
                + points[2].x * (points[0].y - points[1].y)
            )
            / 2
        )

        if area == 0:
            raise ValueError("The points must not lie on one line")

        side_lengths = [
            math.dist((points[0].x, points[0].y), (points[1].x, points[1].y)),
            math.dist((points[1].x, points[1].y), (points[2].x, points[2].y)),
            math.dist((points[2].x, points[2].y), (points[0].x, points[0].y)),
        ]

        rounded_lengths = {round(length, 6) for length in side_lengths}
        if len(rounded_lengths) != 3:
            raise ValueError("The triangle must be scalene")

    @staticmethod
    def _next_screenshot_name() -> str:
        """Return an available screenshot filename in the active directory."""
        screenshot_number = 1

        while True:
            filename = f"{SCREENSHOT_PREFIX}_{screenshot_number}.ps"
            if not Path(filename).exists():
                return filename

            screenshot_number += 1


def main() -> None:
    """Application entry point."""
    TriangleApplication().run()


if __name__ == "__main__":
    main()
```



Задание 2.

Реализовать построение заданного типа фрактала по варианту

Везде, где это необходимо, предусмотреть ввод параметров, влияющих на внешний вид фрактала

4) Ковер Серпинского

```
"""Task 2, variant 4: Sierpinski carpet fractal builder."""

# pylint: disable=too-few-public-methods,duplicate-code

from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
from tkinter import Button, Canvas, Entry, Frame, Label, StringVar, Tk, messagebox

WINDOW_WIDTH = 1000
WINDOW_HEIGHT = 700
CONTROL_WIDTH = 270
CANVAS_WIDTH = WINDOW_WIDTH - CONTROL_WIDTH
CANVAS_HEIGHT = WINDOW_HEIGHT
SCREENSHOT_PREFIX = "sierpinski_carpet"

BACKGROUND_COLOR = "#202124"
CARPET_COLOR = "#40c4ff"
```

```
HOLE_COLOR = "#202124"
PANEL_COLOR = "#303136"
TEXT_COLOR = "#f2f2f2"


@dataclass(frozen=True)
class CarpetParameters:
    """Parameters that affect the fractal appearance."""

    depth: int
    size: int
    offset_x: int
    offset_y: int
    carpet_color: str
    hole_color: str


@dataclass(frozen=True)
class SquareArea:
    """Square area used during recursive fractal building."""

    left: float
    top: float
    size: float


class SierpinskiCarpet:
    """Recursive Sierpinski carpet model."""

    def __init__(self, parameters: CarpetParameters) -> None:
        self.parameters = parameters

    def update_parameters(self, parameters: CarpetParameters) -> None:
        """Apply new fractal parameters."""
        self.parameters = parameters

    def draw(self, canvas: Canvas) -> None:
        """Draw the fractal on the canvas."""
        parameters = self.parameters
        canvas.create_rectangle(
            parameters.offset_x,
            parameters.offset_y,
            parameters.offset_x + parameters.size,
            parameters.offset_y + parameters.size,
            fill=parameters.carpet_color,
            outline=parameters.carpet_color,
        )
        self._cut_center_squares(
            canvas,
```

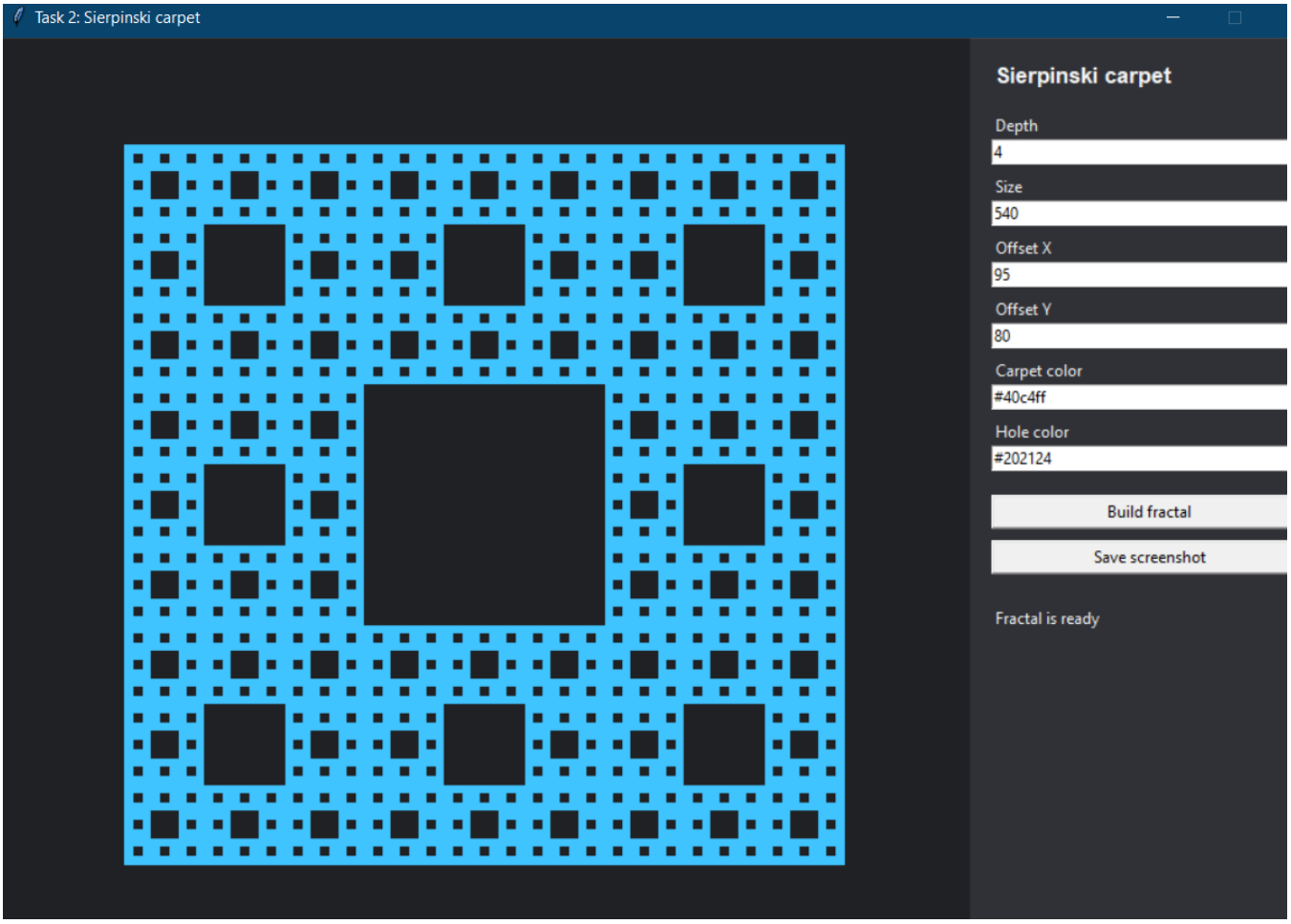
```
            SquareArea(parameters.offset_x, parameters.offset_y, parameters.size),
            parameters.depth,
        )

    def _cut_center_squares(
        self,
        canvas: Canvas,
        area: SquareArea,
        depth: int,
    ) -> None:
        """Recursively remove center squares from the carpet."""
        if depth <= 0:
            return

        next_size = area.size / 3
        center_left = area.left + next_size
        center_top = area.top + next_size

        canvas.create_rectangle(
            center_left,
            center_top,
            center_left + next_size,
            center_top + next_size,
            fill=self.parameters.hole_color,
            outline=self.parameters.hole_color,
        )

        for row in range(3):
            for column in range(3):
                if row == 1 and column == 1:
                    continue

                self._cut_center_squares(
                    canvas,
                    SquareArea(
                        area.left + column * next_size,
                        area.top + row * next_size,
                        next_size,
                    ),
                    depth - 1,
                )


class FractalApplication:
    """Window application for building a Sierpinski carpet."""

    def __init__(self) -> None:
        self.root = Tk()
```

```
        self.root.title("Task 2: Sierpinski carpet")
        self.root.geometry(f"{WINDOW_WIDTH}x{WINDOW_HEIGHT}")
        self.root.resizable(False, False)

        self.entries: dict[str, Entry] = {}
        self.status_text = StringVar(value="Fractal is ready")
        self.carpet = SierpinskiCarpet(
            CarpetParameters(
                depth=4,
                size=540,
                offset_x=95,
                offset_y=80,
                carpet_color=CARPET_COLOR,
                hole_color=HOLE_COLOR,
            )
        )

        self.canvas = Canvas(
            self.root,
            width=CANVAS_WIDTH,
            height=CANVAS_HEIGHT,
            bg=BACKGROUND_COLOR,
            highlightthickness=0,
        )
        self.canvas.pack(side="left", fill="both")

        control_panel = Frame(
            self.root,
            width=CONTROL_WIDTH,
            bg=PANEL_COLOR,
            padx=16,
            pady=16,
        )
        control_panel.pack(side="right", fill="y")
        control_panel.pack_propagate(False)

        self._create_controls(control_panel)
        self.draw()

    def _create_controls(self, control_panel: Frame) -> None:
        """Create input controls for fractal parameters."""
        Label(
            control_panel,
            text="Sierpinski carpet",
            bg=PANEL_COLOR,
            fg=TEXT_COLOR,
            font=("Arial", 13, "bold"),
        ).pack(anchor="w", pady=(0, 14))
```

```
        default_values = {
            "Depth": "4",
            "Size": "540",
            "Offset X": "95",
            "Offset Y": "80",
            "Carpet color": CARPET_COLOR,
            "Hole color": HOLE_COLOR,
        }

        for label_text, value in default_values.items():
            self._add_entry(control_panel, label_text, value)

        Button(
            control_panel,
            text="Build fractal",
            command=self.apply_parameters,
        ).pack(fill="x", pady=(12, 4))
        Button(
            control_panel,
            text="Save screenshot",
            command=self.save_screenshot,
        ).pack(fill="x", pady=4)

        Label(
            control_panel,
            textvariable=self.status_text,
            bg=PANEL_COLOR,
            fg=TEXT_COLOR,
            wraplength=225,
            justify="left",
        ).pack(anchor="w", pady=(18, 0))

    def _add_entry(self, control_panel: Frame, label_text: str, value: str) -> None:
        """Add a labeled entry to the control panel."""
        Label(control_panel, text=label_text, bg=PANEL_COLOR,
fg=TEXT_COLOR).pack(anchor="w")
        entry = Entry(control_panel)
        entry.insert(0, value)
        entry.pack(fill="x", pady=(0, 6))
        self.entries[label_text] = entry

    def apply_parameters(self) -> None:
        """Read screen input and rebuild the fractal."""
        try:
            parameters = self._read_parameters()
            self.carpet.update_parameters(parameters)
            self.draw()
```

```
            self.status_text.set("Fractal rebuilt")
        except ValueError as error:
            messagebox.showerror("Input error", str(error))

    def draw(self) -> None:
        """Draw the current fractal."""
        parameters = self.carpet.parameters
        self.canvas.delete("all")
        self.canvas.configure(bg=parameters.hole_color)
        self.carpet.draw(self.canvas)

    def save_screenshot(self) -> None:
        """Save the current canvas image to the active directory."""
        filename = self._next_screenshot_name()
        self.canvas.postscript(file=filename, colormode="color")
        self.status_text.set(f"Screenshot saved: {filename}")

    def run(self) -> None:
        """Start the application."""
        self.root.mainloop()

    def _read_parameters(self) -> CarpetParameters:
        """Read and validate fractal parameters from entries."""
        depth = int(self.entries["Depth"].get())
        size = int(self.entries["Size"].get())
        offset_x = int(self.entries["Offset X"].get())
        offset_y = int(self.entries["Offset Y"].get())
        carpet_color = self.entries["Carpet color"].get().strip()
        hole_color = self.entries["Hole color"].get().strip()

        if not 0 <= depth <= 6:
            raise ValueError("Depth must be from 0 to 6")

        if size <= 0:
            raise ValueError("Size must be positive")

        if offset_x < 0 or offset_y < 0:
            raise ValueError("Offsets must not be negative")

        if offset_x + size > CANVAS_WIDTH or offset_y + size > CANVAS_HEIGHT:
            raise ValueError("The carpet must fit inside the drawing area")

        if not carpet_color or not hole_color:
            raise ValueError("Colors must not be empty")

        return CarpetParameters(
            depth=depth,
            size=size,
```

```
            offset_x=offset_x,
            offset_y=offset_y,
            carpet_color=carpet_color,
            hole_color=hole_color,
        )

    @staticmethod
    def _next_screenshot_name() -> str:
        """Return an available screenshot filename in the active directory."""
        screenshot_number = 1

        while True:
            filename = f"{SCREENSHOT_PREFIX}_{screenshot_number}.ps"
            if not Path(filename).exists():
                return filename

            screenshot_number += 1


def main() -> None:
    """Application entry point."""
    FractalApplication().run()


if __name__ == "__main__":
    main()
```



В ходе выполнения работы были реализованы две графические программы с использованием библиотеки tkinter.